

This homework has 8 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

(0 pts) 0. **Before you start; before you submit.**

- (a) Before starting this assignment, carefully read [Handout 3](#) about “giving an algorithm,” and follow it in your solutions.
- (b) Correctness arguments and runtime analysis for divide-and-conquer/recursive algorithms should follow the recommended structure described in this week’s discussion worksheet.
- (c) If applicable, state the name(s) and username(s) of your collaborator(s).

Solution:

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn’t show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn’t do last week’s homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

Solution:

Grading notes:

2. **Recurrences.**

Define $T(n) = 9T(n/3) + O(g(n))$. With brief justification, apply the Master Theorem to derive a closed-form asymptotic (big-O) solution for $T(n)$ when $g(n)$ is each of the following:

- (5 pts) (a) $g(n) = n^{3.5}$.

Solution: We apply the Master Theorem with $k = 9, b = 3$ and $d = 3.5$. Since $k = 9 < 3^{3.5} = b^d$, we have that

$$T(n) = O(n^{3.5}).$$

Grading notes:

+2 Correct values for k, b , and d .

+2 Figured out correct case of Master Theorem to apply.

+1 Correct final answer.

(5 pts) (b) $g(n) = n \log n$.

Solution: $g(n) = n \log(n) = O(n \log n)$, so we apply the Master Theorem with log factors with $k = 9, b = 3, d = 1$, and $w = 1$. Since $k = 9 > 3^1 = b^d$, we have that

$$T(n) = O(n^{\log_b k}) = O(n^{\log_3 9}) = O(n^2).$$

(In fact, since $g(n) = \Theta(n \log n)$, the above O -bound on $T(n)$ is a tight Θ -bound as well.)

Grading notes:

+1 Used Master Theorem with log factors (instead of the normal version).

+1 Correct values for k, b , and d .

+2 Figured out correct case of Master Theorem (with log factors) to apply.

+1 Correct final answer.

(5 pts) (c) $g(n) = 5n^2$.

Solution: $g(n) = 5n^2 = O(n^2)$, so we apply the Master Theorem with $k = 9, b = 3$, and $d = 2$. Since $k = 9 = 3^2 = b^d$, we have that

$$T(n) = O(n^d \log n) = O(n^2 \log n).$$

(In fact, since $g(n) = \Theta(n^2)$, the above O -bound on $T(n)$ is a tight Θ -bound as well.)

Grading notes:

+2 Correct values for k, b , and d .

+2 Figured out correct case of Master Theorem to apply.

+1 Correct final answer.

(15 pts) 3. **This is the pits.**

A *pit* of an integer array $M[1, \dots, n]$ is an element $M[i]$ that is no larger than its adjacent elements. More precisely, $M[i]$ is a pit if $M[i] \leq M[i-1]$ and $M[i] \leq M[i+1]$, where for convenience we say that $M[0] = M[n+1] = \infty$ to deal with the endpoints of the array. Note that by this definition, if M has just one element, then that element is a pit. Every array has at least one pit—namely, any smallest element in the array—though there may be others.

A trivial algorithm can find the index of a pit of $M[1, \dots, n]$ in $O(n)$ time. **Give an algorithm** to find the index of a pit in $O(\log n)$ time.

Hint: Think about another $O(\log n)$ -time algorithm you know that operates on one-dimensional arrays.

Note: Make sure you have read [Handout 3](#) before attempting this question, and that you include all of the required parts.

Solution: Our algorithm is inspired by binary search: we first check whether the “middle” element of the array is a pit, and if so return its index; otherwise, we recursively find the index of a pit in *one* of the “left half” or “right half” sub-arrays that is guaranteed to have one. To determine which half to recurse on, the key insight (which requires proof!) is this: if the middle element is larger than the element preceding it, then the left sub-array must have a pit; similarly, if the element following the middle one is smaller, then the right sub-array must have a pit.

We can express this algorithm more precisely in pseudocode:

```
Input: array  $M[1, \dots, n]$  and indices  $1 \leq start \leq finish \leq n$ , where  $M[start-1] \geq M[start]$ 
and  $M[finish] \leq M[finish+1]$ 
1: function FINDPIT( $M, start, finish$ ) // initially call with  $start = 1, finish = n$ 
2:    $\ell \leftarrow finish - start + 1$  // number of elements in the “slice”  $M[start, \dots, finish]$ 
3:   if  $\ell = 1$  then
4:     return  $start$ 
5:    $mid \leftarrow start + \lfloor \ell/2 \rfloor$ 
6:   if  $M[mid-1] \geq M[mid]$  and  $M[mid] \leq M[mid+1]$  then
7:     return  $mid$  // mid element is not larger than either neighbor, so it’s a pit
8:   else if  $M[mid-1] < M[mid]$  then // if mid’s left neighbor is smaller...
9:     return FINDPIT( $M, start, mid-1$ ) // ...recurse to the left
10:  else
11:    return FINDPIT( $M, mid+1, finish$ ) // otherwise recurse to the right
```

Correctness: It will suffice to show three properties: (a) the *initial* call to FINDPIT satisfies the preconditions listed on the Input line of the algorithm; (b) any value *directly* output by the function (i.e., not as the result of a recursive call) is the index of some pit *in the full array* M ; and (c) any *recursive* call to FINDPIT both *satisfies the preconditions* listed on the Input line of the algorithm.

These properties imply correctness because by an inductive argument since (a) and (c) imply that each recursive call is valid, and by (b), any direct output must be the index of a pit of the full array M .

Statement (a) follows immediately from our convention that $M[0] = M[n+1] = \infty$.

For Statement (b), on line 7 the return value is a confirmed pit due to the check on line 6. On line 4, the return value is a confirmed pit *of the entire array* M because the precondition implies that $M[start-1] \geq M[start]$ and $M[start] \leq M[start+1]$ since $start = finish$.

Now we prove statement (c). There are two (mutually exclusive) recursive calls to consider, on lines 9 and 11, which apply only when $\ell \geq 2$ and hence $\lfloor \ell/2 \rfloor \geq 1$. We first check that $1 \leq \text{start} \leq \text{finish} \leq n$ (from the precondition). For the first call, we have that

$$1 \leq \text{start} \leq \text{mid} - 1 \leq \text{start} + (\text{finish} - \text{start} + 1)/2 - 1 = (\text{finish} + \text{start} - 1)/2 < \text{finish} \leq n,$$

as needed. For the second call, even though $\text{mid} \leq \text{finish}$, it is not necessarily the case that $\text{mid} + 1 \leq \text{finish}$. Fortunately, if $\text{mid} + 1 > \text{finish}$ it must be that $\text{mid} = \text{finish}$, hence $M[\text{mid}] \leq M[\text{mid} + 1]$ by the precondition. But based on the comparisons done on lines 6 and 8, line 11 can be reached only if $M[\text{mid}] > M[\text{mid} + 1]$. Therefore, line 11 cannot be reached unless $\text{mid} + 1 \leq \text{finish}$, hence we indeed have $1 \leq \text{mid} + 1 \leq \text{finish} \leq n$.

Finally, we can see that the rest of the precondition is satisfied for both calls. In the first call, $M[\text{mid} - 1] < M[\text{mid}]$ from the check on line 8 (and $M[\text{start} - 1] \geq M[\text{start}]$ from the precondition of the parent call). In the second call, $M[\text{mid}] > M[\text{mid} + 1]$ due to the comparisons on lines 6 and 8 (and $M[\text{finish}] \leq M[\text{finish} + 1]$ from the precondition of the parent call).

Running time: Let $T(n)$ be the running time of the algorithm on an array of n entries. Then it satisfies the recurrence

$$T(n) = T(n/2) + O(1)$$

because it takes $O(1)$ time to read the middle element and compare it to its neighbors, and if we perform a recursive call, we perform it on an array that is at most half the size of the original. Applying the Master Theorem, this recurrence solves to $T(n) = O(\log n)$.

Alternate Solution: It is also ok to remove lines 6-7 and then on line 11 use the subarray from mid to finish instead of $\text{mid} + 1$ to finish (or a logically symmetric construction). In this case, we would also need to have a base case of size 2 to avoid infinite recursion. In general, if the recursion can include mid itself, we need to be careful about infinite recursion on size-2 arrays. Whether or not infinite recursion is possible will depend on whether mid is calculated using a floor or a ceiling, and which side of the recursion includes mid .

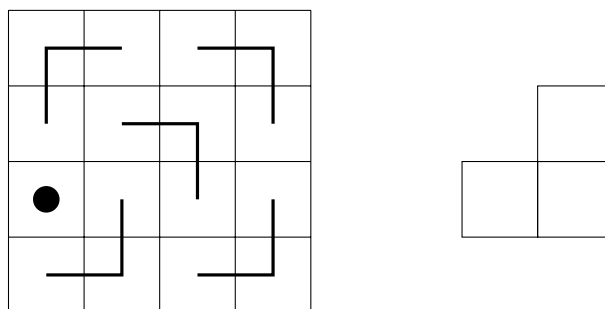
Grading notes:

- +2 Base case(s)
- +1 Cuts array in half
- +1 Makes a recursive call on half of the array
- +3 Recursive calls occur under the correct conditions
- +2 At least a partially correct argument of correctness
- +3 Complete, clear, and correct argument of correctness.
- +2 Correct recurrence for running time analysis
- +1 Justification for running time analysis

4. A hole different kind of tiling.

Consider the following tiling problem: for some $n \geq 2$ that is a power of two, we want to tile an n -by- n grid of unit squares using “L-shaped” tiles that are made up of three unit squares each. However, there is *one designated* unit square in the grid that should remain *uncovered*. The rest should be covered by tiles, without any overlap and with every tile contained entirely within the grid. The tiles may be oriented arbitrarily.

See the diagram below for a small example of a four-by-four grid and the shape of the tile. The uncovered square is marked by a circle, and the thick right angles show the orientations of the tiles in a valid tiling.



It is not immediately obvious that there is a valid tiling for any power-of-two n and desired uncovered square; indeed, certain approaches for placing tiles may “get stuck” in unresolvable configurations. In this problem you will develop and analyze an algorithm that works in all cases.

- (10 pts) (a) Below is an *incomplete* algorithm for this problem, whose missing parts you will provide. The input is a square n -by- n grid where $n \geq 2$ is a power of two, and (the location of) a designated unit square to leave uncovered. The algorithm places tiles accordingly on the grid; this tiling is its “output.”

Provide the missing parts to make the algorithm correct. (You will justify correctness and analyze running time in the subsequent parts.)

Algorithm outline (please fill in the blanks):

Solution:

1. *Base case* (if $n = \underline{\hspace{1cm}}$ [MISSING PART #1]):
[MISSING PART #2]

2. *Recursive case* (if $n > \underline{\hspace{1cm}}$ [MISSING PART #3]):

- Designate three additional unit squares, different from the given one, to leave uncovered for now, so that:

- i. they can be covered by a single tile, and
- ii. each of the four $n/2$ -by- $n/2$ *quadrants* (upper left, lower right, etc.) of the grid has exactly one square to leave uncovered.

Specifically, choose the three squares as follows: [MISSING PART #4]

- [MISSING PART #5]

- Place a tile to cover the three additional unit squares that were designated above.

Solution:

PARTs #1, #2, #3 : Two valid sets of answers, both are acceptable.

First answer:

PART #1: If $n = 1$

PART #2: Do nothing, the single designated square is already uncovered.

PART #3: If $n > 1$

Second answer:

PART #1: If $n = 2$

PART #2: Place one tile, oriented so that the designated square is uncovered (and the other three are covered).

PART #3: If $n > 2$

PART #4: in the *two-by-two subgrid* at the *center* of the grid, choose the three squares from the three quadrants that do *not* contain the already-designated square.

PART #5: recursively tile each quadrant, each with its one designated square to leave uncovered.

Grading notes:

+3 Correct base case (PART 1, 2, and 3)

+4 Correct PART 4

+3 Correct PART 5

- (10 pts) (b) Briefly justify the correctness of the algorithm, as you completed it in the previous part.

Solution: For the base case, there's two possible solutions.

If $n = 1$, then the single square is already uncovered, so leave it as is.

When $n = 2$, there is a two-by-two grid with one unit square to leave uncovered, and

the algorithm covers the other three squares, as needed.

For the recursive case, by the choice of three additional squares to leave uncovered according to (PART #4), we are left with four $n/2$ -by- $n/2$ quadrants, each of which has exactly one square to leave uncovered. By recursion/induction, the algorithm tiles those quadrants correctly, leaving the four designated unit squares uncovered. Finally, by the choice of the three additional squares in (PART #4), they are covered by the one tile as placed by the algorithm. This leaves just the original designated unit square uncovered, as desired.

Grading notes:

- +3 Correct justification for base case
- +3 Correct application of the inductive step to tile the smaller quadrants correctly
- +4 Correctly explains how to tile the remaining designated uncovered squares

- (5 pts) (c) Let $T(n)$ denote the running time of the algorithm as a function of the side length n . The cost to place a tile at a specified location, with a specified orientation, is $O(1)$. Derive, with brief justification, a recurrence and asymptotic closed-form solution for $T(n)$; both should be the tightest possible.

Solution: The recurrence is $T(n) = 4T(n/2) + O(1)$. This is because choosing (and finally covering) the three squares takes $O(1)$ time, since we just need to identify which quadrant the input square belongs to, and we make four recursive calls on subgrids of side length $n/2$.

This recurrence is covered by the Master Theorem, with $k = 4$, $b = 2$, and $d = 0$ (because $n^0 = 1$). Since $k > b^d$, the solution is $T(n) = O(n^{\log_b k}) = O(n^2)$.

All of the above bounds are asymptotically tight, because choosing and covering the three squares also takes $\Omega(1)$ and hence $\Theta(1)$ time, so we get $T(n) = \Theta(n^2)$ in this case.

Grading notes:

- +2 Correct recurrence relation
- +1 Correct justification for recurrence relation
- +1 Correct asymptotic closed-form solution
- +1 Correct justification for asymptotic closed-form solution

- (15 pts) 5. **Hadamard Matrix multiplication.**

The matrix H_t is a $n \times n$ matrix where $n = 2^t$. It is defined recursively, where $H_0 = (1)$, and in general,

$$H_{t+1} = \begin{pmatrix} H_t & H_t \\ H_t & -H_t \end{pmatrix}$$

For example,

$$H_2 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$$

It turns out that this matrix has many [applications](#) including signal processing and quantum computation.

Given a vector $x \in \mathbb{Z}^n$, there is a trivial algorithm that uses $O(n^2)$ operations to compute the matrix-vector product $H_t x$, by first computing the matrix H_t and then computing its product with x . Give an algorithm that computes $H_t x$ in $O(n \cdot t) = O(n \log n)$ operations.

Note: Recall how matrix vector multiplication works. If we have a $n \times n$ matrix A and a vector $x \in \mathbb{Z}^n$, their product is calculated by taking the dot product of x with each row in A . (That

is, if the row is $[a_1, a_2, \dots, a_n]$ and $x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$, then the dot product is $a_1 x_1 + a_2 x_2 + \dots + a_n x_n$).

For example, here we have a 2×2 matrix A and a vector $x \in \mathbb{Z}^2$.

$$Ax = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{bmatrix}$$

Moreover, instead of multiplying each individual row and column, we can also multiply matrices in blocks. For example, if we have a 4×4 matrix A and a vector $x \in \mathbb{Z}^4$, we can break A into 4 smaller matrices, say 2×2 matrices H, I, J, K . We can also break x into 2 vectors $\vec{x}_1$ and $\vec{x}_2$ each of size 2. This would give the following formulation of the product Ax .

$$Ax = \begin{bmatrix} H & I \\ J & K \end{bmatrix} \begin{bmatrix} \vec{x}_1 \\ \vec{x}_2 \end{bmatrix} = \begin{bmatrix} H\vec{x}_1 + I\vec{x}_2 \\ J\vec{x}_1 + K\vec{x}_2 \end{bmatrix}.$$

Solution: The following algorithm recursively computes $H_t x$:

```

Input: A vector  $\vec{x}$  of dimension  $2^t$ 
1: function COMPUTEHX( $\vec{x}$ )
2:    $t := \log_2(\text{the dimension of } \vec{x})$ 
3:   if  $t = 0$  then
4:     return  $\vec{x}$ 
5:   else
6:      $A = \text{COMPUTEHX}(\vec{x}[0 \dots 2^{t-1} - 1])$ 
7:      $B = \text{COMPUTEHX}(\vec{x}[2^{t-1} \dots 2^t - 1])$ 
8:     return  $\begin{bmatrix} A + B \\ A - B \end{bmatrix}$ 
```

Runtime analysis:

Let $T(n)$ be the runtime of COMPUTEHX with input size $n = 2^t$.

Lines 2-3 are $O(1)$.

Lines 6 and 7 are each $T(n/2)$, since we call COMPUTEHX with a vector that is half the size of $\vec{x}$.

Line 8 is $O(n)$, since we add or subtract vectors of half the size of x twice.

Then the recurrence relation is $T(n) = 2T(n/2) + O(n)$.

Using the Master Theorem, since $2/2^1 = 1$, $T(n) = O(n \log_2 n)$.

Correctness Analysis:

For the base case $t = 0$, $\vec{x}$ is a 1×1 vector, and $H_t \vec{x} = [1] \vec{x} = \vec{x}$.

Otherwise,

$$H_t \vec{x} = \begin{bmatrix} H_{t-1} & H_{t-1} \\ H_{t-1} & -H_{t-1} \end{bmatrix} \begin{bmatrix} \vec{x}_1 \\ \vec{x}_2 \end{bmatrix} = \begin{bmatrix} H_{t-1} \vec{x}_1 + H_{t-1} \vec{x}_2 \\ H_{t-1} \vec{x}_1 - H_{t-1} \vec{x}_2 \end{bmatrix}$$

which is exactly what COMPUTEHX returns. $\square$

Grading notes:

- +1 Base case is correctly defined (if $t = 0$ then return input vector).
- +2 Call function recursively on first half of vector.
- +2 Call function recursively on second half of vector.
- +3 return $\begin{bmatrix} A + B \\ A - B \end{bmatrix}$
- +3 Runtime analysis follows correctly from proposed algorithm using the Master Theorem.
If the algorithm's calculated runtime is not $O(n \log n)$, 0 points for this item (due to failure to follow problem).
- +1 Base case is correctly analyzed in correctness proof.
- +3 Recursive case is correctly analyzed in correctness proof.

- (20 pts) 6. **Maize-Blue-Coloring.** Let $G = (V, E)$ be an undirected graph. A *maize-blue* coloring on G is an assignment (denoted by color) from each vertex $v \in V$ to one of the colors maize or blue. Determine if the following algorithm terminates on all inputs. $\square$

```

1: function MAIZE-BLUE-COLORING( $G = (V, E)$ )
   // Initialization
2:   for  $v \in V$  do
3:     Randomly assign either maize or blue to  $v$ 
   // Main loop
4:   while there exists a vertex  $v$  with more than half of  $v$ 's neighbors the same color as  $v$  do
5:     Recolor  $v$  to the opposite color

```

If you think it terminates on every input, use the potential method to prove that the algorithm terminates, where your potential function may reference the vertices and edges of the graph, including information about how the vertices are colored.

If you think it does not terminate on every input, provide a graph and coloring on which the algorithm does not terminate and show the state of the graph at each step of the algorithm until the graph enters a state which it was in previously.

Solution: *Trivial: It turns out that both this problem and the previous problem have another interesting application! They are both used to construct error-correcting codes, which are techniques used to protect against errors arising in imperfect communication channels.*

We will apply a potential argument to prove this algorithm terminates on all inputs.

First, observe that the number of edges with both vertices assigned to the same color decreases by at least one every time a vertex is recolored. To see why, recall that we only recolor vertices v when more than half of v 's neighbors have the same color as v . For a vertex v , denote the number of maize-colored neighbors of v by $m(v)$, and the number of blue-colored neighbors by $b(v)$.

If v is a maize-colored vertex with the above property, then $m(v) > b(v)$. Observe that after recoloring v , the number of same-colored edges (edges having the same color on both endpoints) decreases by at least one. This is because v has $m(v) + b(v)$ incident edges, and $m(v)$ edges are same-colored, and $b(v)$ edges are different-colored. After we recolor v , we have $b(v)$ edges that are same-colored, and $m(v)$ edges that are different colored. Since $m(v) > b(v)$, and because the edges that are not incident to v remain unchanged by v 's recoloring, we see that the number of same-colored edges has decreased by at least one. A similar argument can be made for the case where v is a blue-colored vertex, and $b(v) > m(v)$ was true.

Formally, define the number of edges with both vertices assigned the same color to be the potential of the algorithm. This quantity is initially at most $|E|$ and can be no less than 0. Every recoloring of a vertex causes the potential to decrease by at least 1. Therefore, the algorithm terminates after at most $|E|$ recolorings.

Grading notes:

- +2 Correctly stated that the algorithm terminates.
- +4 Correct potential function.
- +3 Mentions correct lower bound that the potential never goes below.
- +2 Proves the lower bound.
- +3 Mentions that the potential decreases by at least one for each iteration.
- +4 Proves that the potential indeed decreases by at least one.
- +2 Correctly derives an upper bound of at most $|E|$ recolorings.

(3 EC pts) 7. **Optional extra credit: Robot Testing!**

The Robotics Department has n supposedly identical robots that – in principle – are capable of testing each other. The Robotics Department’s test jig accommodates two robots at a time. When the jig is loaded, each robot tests the other and reports whether it is good or bad. A good robot always reports accurately whether the other robot is good or bad, but the answer of a bad robot cannot be trusted - that is, *a bad robot may respond incorrectly or correctly*. For example, if two bad robots are placed in front of each other, it’s possible (though not guaranteed) that the two robots will both falsely state that the other robot is good. Thus, the four possible outcomes of a test are as follows:

Robot A says -----	Robot B says -----	Conclusion -----
B is good	A is good	Both good, or both bad
B is good	A is bad	At least one is bad
B is bad	A is good	At least one is bad
B is bad	A is bad	At least one is bad

Assume that we are guaranteed that *more* than half of the n robots are good. Our ultimate goal is to identify all of the good robots. To do that, we’ll use a divide-and-conquer process to find one good robot. Then, we’ll use that one good robot to test each of the other $n - 1$ robots.

Here’s how the divide-and-conquer step will work: We’ll pair up the n robots arbitrarily (arbitrarily because we don’t yet know anything about which robots are good and which are bad). If n is odd, one robot won’t be paired. Then, for each pair of robots, we’ll see what they say about one another. Based on what they say, we’ll somehow decide which robots to advance to the next phase. Our goal is to advance $n/2$ robots or fewer to the second phase while maintaining the property that more than half of the robots in that second phase are good.¹ Then we’ll do the same thing again. We’ll pair up the robots that made it to the second phase (possibly leaving one unpaired if the number of robots is odd), see what they say about one another, and again somehow select at most half of those robots to go on to the third phase, maintaining the property that more than half of the robots in that phase are good. This is divide-and-conquer because we’re dividing the population of robots by roughly a factor of two and then conquering the set of robots in a smaller set! At the end of this process, we’ll get down to one robot that we know is good and we’ll use that robot to test all of the original $n - 1$ other robots. Snazzy!

- (a) Given a set of n robots, more than half of which are good, explain how $n/2$ pairwise tests can be used to reduce that set to $n/2$ (or perhaps $\frac{n+1}{2}$) or fewer robots where, still, more than half of those robots are good. In your explanation, specify which robots are kept and which are discarded based on the outcomes of the pairwise tests. (Start off by assuming that n is even. Once you’ve worked out that case, come back and work on the case that n is odd.)

Solution:

We’ll begin with the assumption that n is even. A given pair will produce one of the following outputs from a pairwise test: “good-good”, “good-bad” (or vice-versa), or

¹When n is odd, it’s fine if you let $\frac{n+1}{2}$ through to the next round. As long as $n > 1$, this will still reduce the number of robots. For simplicity, you may treat this as $n/2$ in your analysis

“bad-bad”. For every “good-good” pair, we keep one of those two and throw out the other (arbitrarily, since we can’t distinguish these). All other robots are thrown out. (There is also an alternate solution where for “good-bad” pairs, you only throw out the robot that said “good”.) This results in a subset of the original set that is at most half the size of the original because we keep at most one robot from each pair. Now, we must show that in the new subset, more than half of the robots are still good. Here’s why: In the original set, there were more good robots than bad. If a robot is actually good, let’s call it a G-robot and if it’s actually bad let’s call it a B-robot. There must be some G-G pairs by the pigeonhole principle (there are more G’s than B’s, so some G’s must pair up). Moreover, we will throw both robots away from a G-B pair because at least one of those robots will say “bad.” So, the only robots that we will keep are one from every G-G pair and possibly one from each B-B pair (if they both lie and say “good”). But, since there are more G-G pairs than B-B pairs, there will be more good robots kept than bad ones. Thus, in $n/2$ comparisons we have reduced our original set into one that is at most half the size and maintains the property that there are still more good than bad robots.

Next, we turn to the case that n is odd. In this case, one robot is left unpaired in our pairwise testing procedure. If the number of “good-good” outputs is odd, we simply discard the unpaired robot. If the number of “good-good” outputs is even, we keep the unpaired robot in the next iteration of the algorithm. We show that this maintains the invariant that the next set of robots contains more good robots than bad ones.

First, consider the case that we observe an odd number of “good-good” outputs and we denote this odd number by $2k + 1$. Then, there must be either more G-G pairs or more B-B pairs. If there are more B-B pairs then there are at least $k + 1$ B-B pairs and thus at least $2(k + 1) = 2k + 2$ bad robots in that group and, thus, at most $2k$ good robots in that group. Thus, the number of bad robots exceeds the number of good robots by at least 2. Even if the unpaired robot is good, this still results in more bad robots than good robots in total, which is a contradiction. Thus, there must be more G-G pairs than B-B pairs in this group. Therefore, discarding the unpaired robot maintains the invariant.

Finally, consider the remaining case that the number of “good-good” outputs is even and denote this even number by $2k$. At least k pairs must be G-G, since otherwise there are more bad robots than good robots in total. If there are exactly k G-G pairs, then the unpaired robot must be good in order for there to be more good robots than bad. And, if there are more than k G-G pairs then the number of good robots in that set exceeds the number of bad robots by at least $2(k + 1) - 2(k - 1) = 4$ and thus the invariant is maintained even if the unpaired robot is bad.

Grading notes:

+0.5 Correctly keeps one robot from each G-G pair.

+1 Correctly discards both robots away from a G-B pair.

+0.5 Correctly handles the case that n is odd.

- (b) Using the result of part (a) above and still assuming that more than $\frac{n}{2}$ of the robots are good, give the tightest asymptotic bound for the worst case number of comparisons this algorithm must make to determine whether each robot is good or not.

Solution: We repeat the process above until we are left with one good robot. Then, we use that robot to test all of the other robots. The recurrence relation for the worst case number of comparisons that we need to make to find one good robot is $T(n) = T(n/2) + n/2$, which solves to $O(n)$ using the Master Theorem. Testing the other robots requires $n - 1$ additional comparisons, so the total remains $O(n)$.

Grading notes:

+1 Gets $O(n)$ using correct reasoning, e.g. Master Theorem (specifically $T(n) = T(n/2) + n/2$).